

NeuraGate: Service Mesh & API Gateway

Author: Sahil Janjua

Role: Software Engineer / Solutions Architect

1. Executive Summary

NeuraGate is an enterprise-grade, AI-ready API Gateway and Service Mesh built to handle high-throughput, mission-critical traffic. Designed with a "fail-forward" philosophy, it acts as the centralized nervous system for microservices architectures. By combining reactive programming paradigms (Spring WebFlux) with distributed telemetry and autonomous resilience patterns, NeuraGate doesn't just route traffic—it actively defends the system against cascading failures.

2. The Core Use Case & Vision

Modern microservices ecosystems suffer from a common vulnerability: when one downstream service degrades, the entire system can stall. The use case for NeuraGate is to sit between client applications and downstream microservices to provide:

- **Intelligent Routing:** Dynamically directing traffic based on real-time configurations.
- **Zero-Trust Security:** Centralized JWT authentication and Role-Based Access Control (RBAC).
- **Deep Observability:** Extracting metrics from every request to understand system health.
- **Autonomous Resilience:** Automatically cutting off failing services and gracefully shedding load before catastrophic crashes occur.

3. High-Level Architecture

The system is entirely asynchronous and non-blocking, built on **Java 21**, **Spring Boot 3.x (WebFlux)**, and **Project Reactor**.

The infrastructure stack relies on:

- **Redis:** Backs the distributed rate limiter and dynamic route definitions.
- **Apache Kafka:** Acts as the high-throughput message broker for the telemetry pipeline.
- **Apache ZooKeeper:** Manages Kafka cluster state.
- **Prometheus:** Scrapes gateway metrics at regular intervals.
- **Grafana:** Visualizes the metrics scraped by Prometheus.
- **Docker:** Containerizes the entire environment for reproducible, infrastructure-as-code deployments.

4. Component Deep Dive

A. The Main Service (The Gateway)

The Gateway is the entry point. It matches incoming requests against predicates (paths, headers) and applies a chain of filters before forwarding. Because it uses Virtual Threads and asynchronous I/O, it can efficiently handle thousands of concurrent connections using a minimal number of operating system threads.

B. The Mock Service (Inventory)

To rigorously test the gateway, we built a mock Inventory microservice directly into the ecosystem. This isn't just a dummy API; it's a built-in Chaos Engineering target. It features endpoints specifically designed to simulate real-world disasters:

- `/api/inventory/slow`: Introduces artificial latency to simulate database bottlenecks.
- `/api/inventory/flaky`: Randomly returns HTTP 500 errors to simulate application instability.

We use this mock service to physically prove that our gateway's resilience mechanisms actually work under duress.

C. The Telemetry Pipeline (Kafka)

Every request passing through the gateway generates a telemetry event (latency, path, status, caller). Instead of blocking the request thread to log this, we fire-and-forget these events into an **Apache Kafka** topic. This decouples the hot path (routing) from the cold path (analytics). A background Kafka Consumer then ingests this data, storing it in an in-memory MetricsBuffer to calculate live rolling averages and error rates.

D. The Observability Stack (Prometheus & Grafana)

We expose a `/actuator/prometheus` endpoint that formats our internal metrics (JVM memory, CPU, request counts, route metrics, and circuit breaker states). **Prometheus** continuously polls this endpoint every 15 seconds. **Grafana** then sits on top of Prometheus, providing a visually digestible, real-time command center for operations teams.

5. The "Fight Back" Strategy (Resilience & Edge Cases)

The core philosophy of NeuraGate is that downstream services *will* fail. Here is how the gateway "fights back" to protect the broader ecosystem:

Circuit Breakers (Resilience4j)

- **The Concept:** If a service (like our Mock Inventory) starts failing or responding too slowly, continuing to send traffic to it will exhaust gateway resources.
- **The Edge Case:** "Cascading Failures" — where one slow database taking 30 seconds to respond locks up all connection threads across the entire network.
- **The Fight Back:** We implemented Sliding Window Circuit Breakers. If the failure rate

exceeds 50% over a rolling window, the breaker "Opens." All subsequent requests are immediately blocked at the gateway with an HTTP 503 (or routed to a fallback payload) *without* waiting for a timeout.

Distributed Rate Limiting (Redis)

- **The Concept:** Protecting APIs from DDoS attacks or noisy neighbor microservices.
- **The Edge Case:** A misconfigured client script goes rogue and blasts the `/api/inventory` endpoint with 10,000 requests per second.
- **The Fight Back:** Using a Redis-backed Token Bucket algorithm, we strictly enforce rate limits (e.g., 20 requests per second per user). Excess requests are instantly flatlined with an HTTP 429 (Too Many Requests).

Fallbacks & Retries

- **The Concept:** Masking temporary network blips from the end-user.
- **The Edge Case:** Packets drop during a deployment swap.
- **The Fight Back:** The gateway automatically retries idempotent (GET) requests. If the service is truly unavailable, the gateway intercepts the failure and returns a cached or static localized JSON payload, rather than exposing raw stack traces to the internet.

6. Live Dashboard & AI Integration (Future Proofing)

We built a custom Live Dashboard served directly from the gateway. It uses Long Polling and Server-Sent Events (SSE) to stream live request feeds and latency metrics straight to the browser without requiring a page refresh.

Furthermore, the telemetry data aggregated in the `MetricsBuffer` is fed into the `AiAdvisorService`. This service acts as an autonomous operator, diagnosing latency spikes or error trends and instantly suggesting architectural remedies.

7. Conclusion

NeuraGate is not just a router; it is an active participant in system stability. By uniting Spring WebFlux, Kafka, and Resilience4j, we have created an architecture that expects failure, monitors it meticulously via Prometheus/Grafana, and autonomously defends itself when disasters occur.